

Detection of SQL Antipatterns in jOOQ Database Access Code Using Large Language Models

Master’s thesis problem statement

Kristo Isberg

Erki Eessaar

Supervisor

9th May 2026

1 Background

In this section, we introduce several concepts related to the topic of the thesis. In Section 1.1 we describe the history of SQL antipatterns and their potential impact on software systems. In Section 1.2 we introduce the Java database access library jOOQ Object Oriented Querying (jOOQ). Finally, in Section 1.3 we explore different methods of antipattern detection using static analysis, both in the context of SQL antipatterns and in broader terms.

1.1 SQL Antipatterns

SQL (Structured Query Language) antipatterns refer to frequently occurring, counterproductive solutions in database schema design or query construction that initially appear effective but ultimately result in poor performance, maintainability challenges, or compromised data integrity. The term “antipattern” was coined by Koenig in 1995 [1, p. 46] and is generally used to describe common solutions to recurring problems that prove to be ineffective and have negative consequences in the long term [2, p. 1], [3, p. 4], [4, p. 6].

A related term, “code smell”, was coined by Beck and Fowler in 1999 [5, Ch. 3]. The term is used to describe characteristics of source code that hint towards the use of bad practices in the code [5, pp. 71–72]. In the context of this thesis, we consider the terms “antipattern” and “code smell” as rough equivalents, as many works on SQL antipatterns and code smells use these terms interchangeably, and their detection methods are generally very similar.

SQL antipatterns were first described by Karwin in a book originally published in 2010 [2], which details “the most frequent missteps software developers naively make while using SQL” [2, p. 1]. Since the publication of the book by Karwin, now considered seminal work on the subject [6, p. 336], researchers and industry professionals alike have produced numerous other collections and taxonomies of SQL antipatterns, both general [7], [8], [9], [10] and database vendor specific [11]. SQL antipatterns can have a negative impact on various characteristics of affected systems, such as performance, maintainability, portability, and data integrity [7, p. 4].

As an example, the “Implicit Columns” antipattern [2, Ch. 19] occurs when a SELECT statement does not specify the columns that are necessary to be selected but rather selects them all using the * wildcard. This degrades the maintainability of the affected system, as several database table operations (such as dropping a column) can change the ordinal positions of columns in the table. If code dependent on previously correct ordinal positions selects its data using the * wildcard after the table layout has changed, it will receive columns in altered ordinal positions, resulting in faulty behaviour.

Furthermore, the antipattern hinders performance, as unneeded columns must be processed by the database management system (DBMS) and the corresponding data is transferred to the affected system. This claim was confirmed by Lyu et al. [12, p. 60], who studied the performance impact of SQL antipatterns in the context of local SQLite databases of Android applications. They found that on average, fixing the antipattern reduced the runtime of affected queries by 27%, and their energy consumption by 29%.

The antipattern can also occur in the form of an INSERT statement, which does not specify the list of columns to which the inserted values should be assigned, but depends on the implicit column order of the table [2, Ch. 19]. An example of an SQL query containing the “Implicit Columns” antipattern is shown in Listing 1.

```
SELECT * FROM employee WHERE name = 'Jack'
```

Listing 1: A query written in SQL containing the “Implicit Columns” antipattern [2, Ch. 19]

1.2 jOOQ

jOOQ Object Oriented Querying (jOOQ)¹ is an open-source Java library that aims to simplify and enhance the way developers interact with SQL databases. Rather than abstracting away SQL (like Object-Relational Mapping (ORM) frameworks such as Hibernate² do), jOOQ integrates SQL into Java as an embedded domain-specific language (DSL) [13]. jOOQ allows developers to build SQL queries using its DSL and execute them, leveraging code generation for enhanced type safety and Integrated Development Environment (IDE) support [14]. Their code generator reverse-engineers an existing database schema into a set of Java classes that model numerous database objects, such as tables, sequences, stored procedures, and user-defined types [14]. For situations where building SQL queries with jOOQ is not preferred, jOOQ allows executing plain SQL statements using Java Database Connectivity (JDBC), acting as an object-oriented convenience wrapper for the rather low-level JDBC API [15].

A query written using the jOOQ DSL containing the “Implicit Columns” antipattern [2, Ch. 19], equivalent to the SQL query in Listing 1, is shown in Listing 2.

```
var employee = dslContext.select(DSL.asterisk())
    .from(EMPLOYEE)
    .where(EMPLOYEE.NAME.eq("Jack"))
    .fetchOne();
```

Listing 2: A query written in jOOQ containing the “Implicit Columns” antipattern [2, Ch. 19]

The approach jOOQ takes to integrate SQL into Java makes it a popular choice for performing complex database interactions. It is one of the most widely used methods for interacting with databases in Java projects, having more than 6,500 stars on GitHub as of October 2025 [16], and being widely adopted in industrial software [17]. It is widely used both as the sole database interaction method and with an ORM such as Hibernate, to perform the most complex and critical database interactions, which ORMs struggle to perform efficiently, if they can perform them at all [18, p. 423].

1.3 Static Analysis for Antipattern Detection

Historically, the detection of antipatterns and code smells has been the domain of traditional static analysis methods, which take metrics-based and rule-based approaches, represented by tools such as SonarQube³ and PMD⁴.

In metrics-based approaches, code smells are detected based on threshold values on software metrics, such as lines of code, complexity, and coupling. For example, a class with high complexity, low cohesion, and any direct access to foreign data could be identified as the “God Class”, also known as “Blob” code smell [19, Ch. 3.3], [4, pp. 73–84], [20, p. 355].

In rule-based approaches, code smells are detected by searching the Abstract Syntax Tree (AST) of the source code for suspicious patterns. For example, a class, which has a procedural name and a very high number of lines of code, has no parameters, uses global variables, and uses neither inheritance nor polymorphism, could be identified as the “Spaghetti Code” smell [4, pp. 119–137], combining rule- and metrics-based approaches [21, p. 27].

Although static analysis tools that use traditional methods have been effective in detecting code smells and antipatterns to an extent, they suffer from notable limitations due to the rigidity of the underlying methods. The metric- and pattern-based tools lack contextual awareness, making them prone to high false positive rates, often reporting issues, which are not actual problems upon human review, which reduces the practical usability of these tools and developers’ confidence in them.

To overcome the limitations of rigid thresholds and patterns, researchers have explored Machine Learning (ML) techniques for code smell detection. Rather than defining fixed thresholds or patterns, these approaches learn from data, such as human-annotated datasets of smelly and clean

¹<https://www.jooq.org/>

²<https://hibernate.org/orm/>

³<https://www.sonarsource.com/products/sonarqube/>

⁴<https://pmd.github.io/>

code. While recent approaches based on Deep Learning show promising results in terms of detection accuracy [22, pp. 3492–3494], traditional techniques remain prominent in real-world scenarios.

More recently, with the rapid evolution of large language models (LLMs), they are being increasingly used to detect problematic patterns in code. LLMs have repeatedly shown advanced semantic understanding and contextual awareness in software engineering tasks [23], [24]. Commercial tools such as CodeRabbit⁵ are widely used to detect issues during code reviews, and various research has indicated that LLMs can detect code quality issues with reasonable accuracy [25], [26], [27], [28]. Most relevantly to our work, Sousa et al. [29, p. 410] found that both small and large language models are reasonably capable of detecting bad smells in PL/SQL code. It should be noted that the models used in many of these studies are considered outdated or deprecated by now, and repeating these studies with recent models would likely demonstrate improved performance.

2 Motivation

Muse et al. [6] studied the prevalence of SQL antipatterns in data-intensive Java software systems and found that SQL antipatterns are prevalent in all studied application domains. They also found that compared to traditional code smells [5, Ch. 3], SQL antipatterns tend to remain unfixed for a longer period and are more likely to persist throughout the lifetime of the software. They suggested that the reason for this could be developers’ lack of awareness of SQL antipatterns and their downsides, or the comparatively low priority of the task of fixing antipatterns.

Both of these reasons could be attributed to the lack of SQL antipattern detection tools, which could bring attention to the occurrences of SQL antipatterns upon their introduction. To the best of our knowledge, there currently exist two static analysis tools capable of detecting SQL antipatterns in Java code.

The first tool, *DbDeo*⁶, was presented by Sharma et al. [30] and is capable of detecting nine schema-related antipatterns in SQL statements embedded in numerous programming languages. They employ regular expressions to extract SQL statements from code written in a host language. The extracted SQL statements are parsed and converted into a model of the statement, which is then analysed by their smell detector. In their accuracy assessment, *DbDeo* demonstrated a low rate of false positives, all of which they attributed to errors during the extraction and parsing phases.

The second tool was presented by Nagy and Cleve [31] and is capable of detecting four of the query-related antipatterns described by Karwin [2]. Their tool uses intra-procedural string resolution to extract SQL statements from Java code, which are then parsed using a fault-tolerant SQL parser, capable of handling SQL statements with incomplete sections due to extraction errors. The tool can be accessed using a command-line interface or the *SQLInspect* plug-in for the Eclipse IDE [31] [32].

As the recommended method of constructing SQL statements with jOOQ is to use its DSL with code generation [33], most statements in projects using jOOQ are not in the form of plain SQL and the SQL is generated dynamically at runtime. Since both of the mentioned tools are designed to work with plain SQL statements, which must be present within the source code, they are incapable of detecting most SQL antipatterns in such projects. Considering the high potential negative impact of SQL antipatterns [7, p. 4], [12] and their wide prevalence in data-intensive software systems [12], we see the need for research around SQL antipatterns in jOOQ.

3 Research Goal

The primary goal of this research is the development of an LLM-based static analysis tool for detecting SQL antipatterns in database access code using jOOQ. These results can be used by developers using jOOQ to detect SQL antipatterns in their codebases, as well as tool developers looking to build tools around this topic.

We focus our research on LLM-based static analysis, rather than alternatives, because of their advanced understanding of code semantics and context, which is highly desirable. Some SQL antipatterns are highly context-specific, and detecting them in jOOQ may require analysing multiple files and having an understanding of the relationships between them. The jOOQ API is also broad, and the dynamic nature of it means that SQL antipatterns can occur in many variations, making traditional static analysis even more ineffective. To the best of our knowledge, we are the first to

⁵<https://www.coderabbit.ai/>

⁶<https://github.com/tushartushar/DbDeo>

approach LLM-based antipattern/code smell detection as a localisation task, rather than a pure classification task, and to evaluate it as such.

Our secondary goal is to study the prevalence of SQL antipatterns in code that uses jOOQ and the specific manifestations of these antipatterns. These results can be used by developers who use jOOQ to gain awareness of antipattern-prone jOOQ APIs, aiding them in “Avoiding the Pitfalls of Database Programming” [2]. Furthermore, these results can be used as input by the jOOQ maintainers to improve the documentation of such APIs or deprecate them in favour of safer alternatives.

- **RQ1:** How do different LLMs compare in identifying SQL antipatterns in Java code using jOOQ for database access?
- **RQ2:** How do different prompting strategies affect LLMs’ accuracy in identifying SQL antipatterns in Java code using jOOQ for database access?
- **RQ3:** How accurate is the developed LLM-based tool in detecting SQL antipatterns in Java code using jOOQ for database access?
- **RQ4:** What are the occurrence patterns of SQL antipatterns in Java code using jOOQ for database access?
- **RQ5:** Which jOOQ API methods are most frequently associated with SQL antipattern occurrences?

4 Research Design

This research follows the principles of Design Science Research [34]. The primary design artefact produced by this research is an LLM-based SQL antipattern detection tool for jOOQ (an instantiation [35, p. 258]).

Additionally, we produce a human-labelled dataset of SQL antipattern occurrences in Java code using jOOQ. Moreover, we perform a large-scale analysis of the prevalence of SQL antipatterns in about 1,000 existing Java software projects using jOOQ.

The research process is divided into five steps. In Section 4.1, we mine software repositories for Java projects using jOOQ. In Section 4.2, we establish ground truth for the development and validation of our tool. In Section 4.3, we perform a series of experiments to decide on a model and prompting strategy to be used in our tool. In Section 4.4, we develop and evaluate the developed tool. Finally, in Section 4.5, we perform a large-scale analysis regarding the occurrence patterns of SQL antipatterns in Java projects using jOOQ.

4.1 Repository Mining

We begin by mining software repositories for projects, used later for evaluation and analysis. We aim to collect as many projects as possible within reason, matching the following criteria:

- contains Java code using jOOQ for database access,
- stores jOOQ generated classes as part of the source code, rather than treating them as derived artefacts, which are not checked into version control [36].

To achieve this goal, we utilise the GitHub code search API [37], using it to search for projects using jOOQ. The GitHub search API has a hard limit of 1,000 results for each search term [37], which requires us to minimise the number of duplicated results for each search term. For example, this means that we cannot search for import statements for jOOQ classes within Java files, as a few large projects could saturate the result list by themselves.

To find the largest number of unique projects possible, we limit our search to the manifest files of projects, looking for statements, which declare a dependency on jOOQ for the project. We limit the search to the manifest file formats of the two most popular build tools for Java [38]: Maven⁷ (e.g. pom.xml) and Gradle⁸ (e.g. build.gradle and build.gradle.kts). We design the search terms to be as specific as possible to keep the number of results below 1,000 where possible. For some of the search terms, we cannot prevent the limit of 1,000 from being exceeded, in which case we

⁷<https://maven.apache.org/>

⁸<https://gradle.org/>

retrieve the 1,000 results that were indexed by the GitHub search infrastructure the most recently, omitting the oldest and most inactive projects.

We further filter the gathered repositories by verifying that the two previously set criteria are met by making sure that the repositories contain jOOQ generated classes as well as other Java files with references to jOOQ classes. We expect the final number of projects to be near 1,000.

We will use a subset of the gathered repositories to establish ground truth in Section 4.2. We will later use the entire collection of repositories to perform a large-scale project analysis in Section 4.5.

4.2 Establishment of Ground Truth

To validate the accuracy of our developed tools, we require a source of ground truth. As SQL antipatterns in jOOQ are an unexplored area, to the best of our knowledge, there are no publicly available datasets of SQL antipatterns in jOOQ available. Furthermore, there are no examples available in the scientific literature on how SQL antipatterns may occur in jOOQ, and the examples are also scarce elsewhere. For example, a remark in the jOOQ manual [39] shows that the “Implicit Columns” antipattern [2, Ch. 19] can occur in jOOQ in unexpected ways. Therefore, examples of antipatterns in plain SQL cannot be solely relied upon to detect SQL antipatterns in jOOQ.

To establish the ground truth, we manually annotate a subset of the projects gathered in Section 4.1. Although the exact number of annotated projects depends on the number of projects in total, we annotate a random subset of projects, which we expect to contain 5–10% of total Java source files containing references to jOOQ APIs, hence possibly containing SQL antipatterns.

Based on prior experience with jOOQ and awareness of SQL antipatterns, we carefully annotate found antipattern occurrences in the code. We establish firm guidelines for annotations to ensure consistency regarding semantics of antipatterns and line spans of antipattern occurrences. We employ a two-annotator setup, where the first annotator (the student) annotates the entire subset of projects and the second annotator (the supervisor) verifies a subset of the annotations, consisting of annotations considered borderline by the first annotator, and random annotations. We assess inter-annotation agreement on antipattern presence using Cohen’s kappa, and agreement on the line spans of antipatterns using Intersection over Union (IoU).

Although more extensive catalogues of SQL antipatterns have been produced, we limit our efforts to the antipatterns defined by Karwin [2], which is considered a seminal work on the subject [6, p. 336] and have the most data available for comparison. We exclude antipatterns, which are undetectable solely from database access code, such as “Pseudokey Neat-Freak” [2, Ch. 22] and “Diplomatic Immunity” [2, Ch. 24], which are rooted in organisational culture.

We divide the set of annotated projects into development, validation, and test sets. We will use the development set for iterative prompt refinement in Section 4.3, the validation set for benchmarking models and prompting strategies in Section 4.3.3, and the test set for evaluating our developed tool in Section 4.4.1.

4.3 Experimental Setup

To select the most suitable combination of an LLM and a prompting strategy for our antipattern detection tool, we perform a series of experiments.

4.3.1 Selecting Models

First, we select a number of language models for our experiments, based on criteria selected to aid us in the development and analysis process, while keeping the selection of models diverse. In particular, we look for the following characteristics in the models:

- **Architectural diversity:** We want to include models with a diverse set of characteristics, such as different model sizes and reasoning vs. non-reasoning models.
- **Performance:** For a given architecture, the model should demonstrate near-state-of-the-art performance on public benchmarks [40].
- **Cost of use:** As we intend to analyse a large amount of code, the analysis should be performed at a reasonable cost. The model should either be relatively cheap to run or the model should be provided through institutional access.

- **Repeatability:** The weights of the model should be stable, allowing the evaluation and analysis to be repeated in the future. Ideally, the model should have open weights. Alternatively, the model provider should offer point-in-time snapshots of the model.
- **Structured outputs:** The model should reliably support structured outputs, providing responses that reliably adhere to expected JSON Schema. This is desired for the later development of the antipattern detection tool to minimise the risk of misformatted responses.

4.3.2 Designing Prompts

We design a number of prompts, representative of different prompting strategies, gathered from recent research on LLMs. We limit our experiments to prompting techniques that can be fully executed in the span of one prompt and one response. Specifically, we focus on the following prompting strategies:

- **Zero-Shot Prompting:** Providing a model with a direct query or task instructions, without providing examples of correct answers in the prompt, leveraging the model’s generalisation capabilities [41].
- **Few-Shot Prompting:** Supplementing the prompt with a number of input-output examples, helping the model recognise patterns [42].
- **Few-Shot Chain-of-Thought Prompting:** Incorporating logical steps into the input-output examples, guiding the model to work through intermediate reasoning steps before arriving at a final answer [43].
- **Zero-Shot Chain-of-Thought Prompting:** Guiding the model to work through intermediate reasoning steps before arriving at a final answer by using a zero-shot prompt with the words “Let’s think step by step.” appended to it [41].
- **Meta Prompting:** Guiding the model by emphasising the structure and reasoning process of tasks, rather than relying on example content [44].
- **Tree-of-Thought Prompting:** Expands on Zero-Shot Chain-of-Thought Prompting by providing the model with an opportunity to explore multiple reasoning branches and to continuously self-evaluate [45], applying concepts from more complex Tree-of-Thought frameworks [46], [47] at prompt level.

We iteratively develop and fine-tune prompts for each technique, comparing the results against the development set produced in Chapter 4.2. For prompting techniques requiring concrete examples (Few-Shot Prompting and Few-Shot Chain-of-Thought Prompting), we acquire the examples from the development set. We test each iteration of our prompts against all tested models until all models reach near-optimal performance. To simplify the experimental setup and the interpretation of results, we only develop universal, model-agnostic prompts, without fine-tuning them for individual models.

4.3.3 Quantitative Analysis

After we finalise the design of all the prompts, we perform an analysis on each of the projects in the validation set produced in Chapter 4.2 using each model and prompting strategy combination. We then evaluate the results of each combination against the ground truth.

In essence, we are evaluating the performance of the models and prompts in performing a **multi-label** (there are multiple antipatterns that can be detected) **multi-occurrence** (each antipattern can occur multiple times within each file) **localisation** (in addition to classification, we aim to detect the location of the antipattern occurrences) task. For this reason, we employ a combination of spatial overlap criteria and classification metrics for evaluation, utilising Intersection over Union (IoU) and F1-Score.

The first step involves quantifying the localisation accuracy using IoU. We define a line span S as the set of line indices within an inclusive range in a source code file.

$$S = \{i \in \mathbb{N} \mid start \leq i \leq end\}$$

For a predicted line span S_p and a ground truth line span S_g , the IoU is defined as:

$$\text{IoU} = \frac{|S_p \cap S_g|}{|S_p \cup S_g|}$$

This metric produces a value between 0 and 1, where 1 represents an exact line-level match. In this methodology, the IoU serves as the qualification gate for all subsequent metrics. A prediction is classified as a True Positive (TP) if its spatial overlap with the ground truth exceeds a pre-defined confidence threshold (typically $\text{IoU} \geq 0.5$). If the IoU is below this threshold, the prediction is classified as a False Positive (FP).

Using this qualification gate, we classify the predictions into True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN), employing Non-Maximum Suppression (NMS): in cases of multi-occurrence where multiple predictions overlap a single ground truth span, only the prediction with the highest confidence score is assigned as a TP, while subsequent overlaps are penalised as FPs. The precision (P) and recall (R) for the selected confidence threshold are defined as:

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad R = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Using the precision and recall, we calculate the F1-Score, which we use to evaluate the performance of the model at the selected threshold:

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$

We calculate P , R and F_1 for each antipattern separately, and as a weighted mean of all antipatterns:

$$\bar{x}_w = \frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i}$$

This way, we are provided with an understanding of which models and prompts are the best in the detection of antipatterns in general, but we also have the opportunity to study the differences in further detail. Based on the results of this analysis, we can provide answers to **RQ1** and **RQ2**, and we can decide on the preferred model and the prompting strategy for the development of an antipattern detection tool in Section 4.4.

4.3.4 Qualitative Analysis

We select a systematic, non-exhaustive set of model and prompting strategy combinations including, but not limited to, the best and worst performers and anomalous outliers. We analyse a representative hand-picked set of code samples using each of the combinations. We study the results of these combinations comparatively and qualitatively in several areas, such as:

- reasoning quality across prompting strategies,
- common FP and FN patterns across models,
- differences in localisation behaviour.

4.4 Development of an Antipattern Detection Tool

We develop an SQL antipattern detection tool to employ the combination of the best-performing model and prompting strategy from Section 4.3.3 for the detection of real-world antipatterns. At minimum, the tool offers the following configuration options, granular enough to facilitate the analysis of the tool and future integrations (e.g. extensions for IDEs), while offering user-friendly defaults for command-line usage:

- **Detection mode:** Allows to switch the granularity of detection between files (“classification”) and line spans (“localisation”), defaults to “localisation”.
- **Output mode:** Allows to switch the output format of the results between machine-readable JSON (“json”) and “human-readable” representations, defaults to “human-readable”.
- **Model used:** Allows users to upgrade the model in the future, defaults to the best-performing model from Section 4.3.3.

4.4.1 Quantitative Analysis

To validate the accuracy of the finalised tool, we perform an analysis on each of the projects in the test set produced in Section 4.2, using the tool in the default “localisation” mode. As in Section 4.3.3, we evaluate the tool using metrics suitable for a multi-label multi-occurrence localisation task: IoU and F1-Score.

So far, we are treating the task as a localisation task, and validating it as such, however there is not much comparison data available for such an approach. To compare our results against other works regarding detecting antipatterns and code smells using LLMs, we also evaluate it as a multi-label classification task.

To achieve this, we re-analyse the projects in the test set, this time using the tool in the “classification” mode, and compare the results against the ground truth. In this case, a prediction is considered a TP, if the tool detects an antipattern within a file, and the ground truth also contains an antipattern within that file.

While performing these calculations, we only include such files from the test set, which only contain up to one occurrence of each antipattern in the ground truth. This is because most other works on code smell detection perform their evaluations on datasets of code samples, which do not contain multiple occurrences of the same antipattern. By doing this, we keep the comparison more fair, as having multiple instances of the same antipattern in a file is likely to increase the number of TPs and decrease the number of FNs.

We compare our accuracy metrics against previous works regarding both SQL antipattern detection [29], [31] and LLM-based code smell detection [25], [26], [27], [28].

4.4.2 Qualitative Analysis

We manually inspect a representative subset of antipattern occurrences detected in the test set to assess whether the detections are semantically correct and aligned with the antipattern definitions established by Karwin [2]. We qualitatively analyse the FPs and FNs present in the subset, discussing the root causes based on the reasoning data provided by the underlying model.

We also compare the outputs of the “localisation” and “classification” modes. We assess the differences in the results, looking for common errors in localisation behaviour and their causes, explaining why classification metrics may appear stronger than localisation metrics.

We conduct a series of semi-structured interviews with developers to assess the perceived usefulness and interpretability of the tool in practice, and to gather suggestions for further enhancements of the tool. We perform the interviews with 5–10 software developers of varying experience, who have current or previous experience with Java, SQL/ORMs, and preferably jOOQ. Topics of the interviews include, but are not limited to, the semantic correctness of the detected antipatterns and the usefulness of localisation in taking action regarding the detections.

Based on the results of the quantitative and qualitative analysis, we can provide an answer to **RQ3**.

4.5 Project Analysis

We use the developed antipattern detection tool to detect SQL antipatterns in all the software projects collected in Section 4.1. We use the tool in the default “localisation” mode to gather information about individual antipattern occurrences.

4.5.1 Quantitative Analysis

We group all detected antipattern occurrences by their type, and for each type of smell we compute the average smell density [48, p. 190], which we define as the number of SQL antipatterns detected per 10 SQL statements, similarly to Sharma et al. [30, p. 6].

As the jOOQ API allows SQL statements to be constructed dynamically, it is difficult to count the exact number of statements in the projects accurately. Other works [6], [30], which detect SQL code smells by extracting SQL statements from source code and then parsing them, receive the number of SQL statements as a side product of the extraction step. We provide a rough estimate of the number of SQL statements in the projects by counting the number of references to jOOQ API methods that execute an SQL statement.

We compare these results against other works studying the prevalence of SQL antipatterns, such as [6], [30]. Based on these results, we can provide an answer to **RQ4**.

4.5.2 Qualitative Analysis

We discuss the quantitative analysis results from a qualitative perspective. We assess, whether there are any major discrepancies between the prevalence numbers found by us and prior works, which focused on plain SQL.

If our results are significantly higher for some antipatterns, we look into the major causes of these antipatterns, looking for jOOQ APIs, which are especially antipattern-prone. Where possible, we provide suggestions for improving the API design of jOOQ. For antipatterns where the opposite is true, we look into ways jOOQ API may prevent these antipatterns from occurring as often. Based on this analysis, we can provide an answer to **RQ5**.

References

- [1] A. Koenig, ‘Patterns and antipatterns,’ *J. Object Oriented Program.*, vol. 8, no. 1, pp. 46–48, 1995.
- [2] B. Karwin, *SQL antipatterns, volume 1*, en. Raleigh, NC: Pragmatic Programmers, Jan. 2023, ISBN: 978-1680508987.
- [3] S. W. Ambler, *SIGS: Managing object technology: Process patterns: Building large-scale systems using object technology series number 15* (Managing object technology series), en. Cambridge, England: Cambridge University Press, Oct. 1998, ISBN: 0521645689.
- [4] W. J. Brown, R. C. Malveau, H. W. McCormick and T. J. Mowbray, *AntiPatterns*, en. Nashville, TN: John Wiley & Sons, Mar. 1998, ISBN: 0471197130.
- [5] M. Fowler, *Refactoring* (Addison-Wesley Signature Series (Fowler)), en, 2nd ed. Boston, MA: Addison Wesley, Jan. 2019, ISBN: 978-0134757599.
- [6] B. A. Muse, M. M. Rahman, C. Nagy, A. Cleve, F. Khomh and G. Antoniol, ‘On the prevalence, impact, and evolution of SQL code smells in data-intensive systems,’ in *MSR ’20: 17th International Conference on Mining Software Repositories, Seoul, Republic of Korea, 29-30 June, 2020*, S. Kim, G. Gousios, S. Nadi and J. Hejderup, Eds., ACM, 2020, pp. 327–338. DOI: [10.1145/3379597.3387467](https://doi.org/10.1145/3379597.3387467).
- [7] B. Alshemaimri, R. Elmasri, T. Alsahfi and M. Almotairi, ‘A survey of problematic database code fragments in software systems,’ *Engineering Reports*, vol. 3, no. 10, e12441, 2021. DOI: [10.1002/eng2.12441](https://doi.org/10.1002/eng2.12441).
- [8] E. Eessaar, ‘On the design of base tables in the sql databases of some existing software,’ in *Software Engineering Perspectives in Systems*, R. Silhavy, Ed., Cham: Springer International Publishing, 2022, pp. 309–324, ISBN: 978-3-031-09070-7. DOI: [10.1007/978-3-031-09070-7_26](https://doi.org/10.1007/978-3-031-09070-7_26).
- [9] E. Eessaar, ‘On the naming of database objects in the sql databases of some existing software,’ in *Software Engineering Research in System Science*, R. Silhavy and P. Silhavy, Eds., Cham: Springer International Publishing, 2023, pp. 534–550, ISBN: 978-3-031-35311-6. DOI: [10.1007/978-3-031-35311-6_51](https://doi.org/10.1007/978-3-031-35311-6_51).
- [10] P. Factor, ‘Sql code smells,’ *Red Gate Software Ltd*, 2014.
- [11] J. Angelakos, *PostgreSQL mistakes and how to avoid them*, en. New York, NY: Manning Publications, Jul. 2025, ISBN: 978-1633436879.
- [12] Y. Lyu, A. Alotaibi and W. G. J. Halfond, ‘Quantifying the performance impact of SQL antipatterns on mobile applications,’ in *2019 IEEE International Conference on Software Maintenance and Evolution, ICSME 2019, Cleveland, OH, USA, September 29 - October 4, 2019*, IEEE, 2019, pp. 53–64. DOI: [10.1109/ICSME.2019.00015](https://doi.org/10.1109/ICSME.2019.00015).
- [13] ‘Sql building,’ Accessed: 7th Oct. 2025. [Online]. Available: <https://www.jooq.org/doc/latest/manual/sql-building/>.
- [14] ‘Code generation,’ Accessed: 7th Oct. 2025. [Online]. Available: <https://www.jooq.org/doc/latest/manual/code-generation/>.
- [15] ‘Sql execution,’ Accessed: 11th Oct. 2025. [Online]. Available: <https://www.jooq.org/doc/latest/manual/sql-execution/>.
- [16] ‘Jooq,’ Accessed: 11th Oct. 2025. [Online]. Available: <https://github.com/jooq/jooq>.
- [17] ‘Some of our customers,’ Accessed: 11th Oct. 2025. [Online]. Available: <https://www.jooq.org/customers>.

- [18] V. Mihalcea, *High-Performance Java Persistence*. Vlad Mihalcea, Oct. 2016, ISBN: 978-9730228236.
- [19] A. J. Riel, *Object-oriented design heuristics*. Boston, MA: Addison-Wesley Educational, Apr. 1996, ISBN: 978-0321774965.
- [20] R. Marinescu, ‘Detection strategies: Metrics-based rules for detecting design flaws,’ in *20th International Conference on Software Maintenance (ICSM 2004), 11-17 September 2004, Chicago, IL, USA*, IEEE Computer Society, 2004, pp. 350–359. DOI: [10.1109/ICSM.2004.1357820](https://doi.org/10.1109/ICSM.2004.1357820).
- [21] N. Moha, Y. Guéhéneuc, L. Duchien and A. L. Meur, ‘DECOR: A method for the specification and detection of code and design smells,’ *IEEE Trans. Software Eng.*, vol. 36, no. 1, pp. 20–36, 2010. DOI: [10.1109/TSE.2009.50](https://doi.org/10.1109/TSE.2009.50).
- [22] R. Malhotra, B. Jain and M. Kessentini, ‘Examining deep learning’s capability to spot code smells: A systematic literature review,’ *Clust. Comput.*, vol. 26, no. 6, pp. 3473–3501, 2023. DOI: [10.1007/S10586-023-04144-1](https://doi.org/10.1007/S10586-023-04144-1).
- [23] M. Chen et al., ‘Evaluating large language models trained on code,’ *CoRR*, vol. abs/2107.03374, 2021. arXiv: [2107.03374](https://arxiv.org/abs/2107.03374).
- [24] T. Liu, C. Xu and J. J. McAuley, ‘Repobench: Benchmarking repository-level code auto-completion systems,’ *CoRR*, vol. abs/2306.03091, 2023. DOI: [10.48550/ARXIV.2306.03091](https://doi.org/10.48550/ARXIV.2306.03091). arXiv: [2306.03091](https://arxiv.org/abs/2306.03091).
- [25] A. Gomes, D. Sousa, P. H. M. Maia and M. Paixão, ‘Attentionsmelling: Using large language models to identify code smells,’ in *Proceedings of the 39th Brazilian Symposium on Software Engineering, SBES 2025, Recife, Brazil, September 22-26, 2025*, L. Teixeira et al., Eds., SBC, 2025, pp. 271–281. DOI: [10.5753/SBES.2025.9921](https://doi.org/10.5753/SBES.2025.9921).
- [26] R. Blaauwendraad, ‘Evaluating large language models for code smell detection,’ Bachelor’s Thesis, Radboud University, 2025. Accessed: 16th Dec. 2025. [Online]. Available: https://www.cs.ru.nl/bachelors-theses/2025/Robert_Blaauwendraad_-_1084960___Evaluating_Large_Language_Models_for_Code_Smell_Detection.pdf.
- [27] A. R. Sadik and S. Govind, ‘Benchmarking LLM for code smells detection: Openai GPT-4.0 vs deepseek-v3,’ *CoRR*, vol. abs/2504.16027, 2025. DOI: [10.48550/ARXIV.2504.16027](https://doi.org/10.48550/ARXIV.2504.16027). arXiv: [2504.16027](https://arxiv.org/abs/2504.16027).
- [28] L. L. Silva, J. R. da Silva, J. E. Montandon, M. Andrade and M. T. Valente, ‘Detecting code smells using chatgpt: Initial insights,’ in *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM 2024, Barcelona, Spain, October 24-25, 2024*, X. Franch, M. Daneva, S. Martínez-Fernández and L. Quaranta, Eds., ACM, 2024, pp. 400–406. DOI: [10.1145/3674805.3690742](https://doi.org/10.1145/3674805.3690742).
- [29] V. Sousa, C. Baptista, A. L. Alves and H. Figueirêdo, ‘Effectiveness of small and large language models for pl/sql bad smell detection,’ in *Anais do XL Simpósio Brasileiro de Bancos de Dados, Fortaleza/CE: SBC, 2025*, pp. 399–412. DOI: [10.5753/sbbd.2025.247256](https://doi.org/10.5753/sbbd.2025.247256).
- [30] T. Sharma, M. Fragkoulis, S. Rizou, M. Bruntink and D. Spinellis, ‘Smelly relations: Measuring and understanding database schema quality,’ in *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice, ICSE (SEIP) 2018, Gothenburg, Sweden, May 27 - June 03, 2018*, F. Paulisch and J. Bosch, Eds., ACM, 2018, pp. 55–64. DOI: [10.1145/3183519.3183529](https://doi.org/10.1145/3183519.3183529).
- [31] C. Nagy and A. Cleve, ‘A static code smell detector for SQL queries embedded in java code,’ in *17th IEEE International Working Conference on Source Code Analysis and Manipulation, SCAM 2017, Shanghai, China, September 17-18, 2017*, IEEE Computer Society, 2017, pp. 147–152. DOI: [10.1109/SCAM.2017.19](https://doi.org/10.1109/SCAM.2017.19).
- [32] C. Nagy and A. Cleve, ‘Sqlinspect: A static analyzer to inspect database usage in java applications,’ in *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings, ICSE 2018, Gothenburg, Sweden, May 27 - June 03, 2018*, M. Chaudron, I. Crnkovic, M. Chechik and M. Harman, Eds., ACM, 2018, pp. 93–96. DOI: [10.1145/3183440.3183496](https://doi.org/10.1145/3183440.3183496).
- [33] ‘Jooq as a sql builder with code generation,’ Accessed: 11th Oct. 2025. [Online]. Available: <https://www.jooq.org/doc/latest/manual/getting-started/use-cases/jooq-as-a-sql-builder-with-code-generation/>.

- [34] A. R. Hevner, S. T. March, J. Park and S. Ram, ‘Design science in information systems research,’ *MIS Q.*, vol. 28, no. 1, pp. 75–105, Mar. 2004, issn: 0276-7783.
- [35] S. T. March and G. F. Smith, ‘Design and natural science research on information technology,’ *Decis. Support Syst.*, vol. 15, no. 4, pp. 251–266, 1995. DOI: [10.1016/0167-9236\(94\)00041-2](https://doi.org/10.1016/0167-9236(94)00041-2).
- [36] ‘Code generation and version control,’ Accessed: 2nd Oct. 2025. [Online]. Available: <https://www.jooq.org/doc/latest/manual/code-generation/codegen-version-control/>.
- [37] ‘Rest api endpoints for search,’ Accessed: 15th Dec. 2025. [Online]. Available: <https://docs.github.com/en/rest/search/search?apiVersion=2022-11-28>.
- [38] ‘Technology — 2025 stack overflow developer survey,’ Accessed: 15th Dec. 2025. [Online]. Available: <https://survey.stackoverflow.co/2025/technology>.
- [39] ‘Sql: Select *,’ Accessed: 29th Sep. 2025. [Online]. Available: <https://www.jooq.org/doc/latest/manual/reference/dont-do-this/dont-do-this-sql-select-all/>.
- [40] ‘Ai model & api providers analysis,’ Accessed: 16th Dec. 2025. [Online]. Available: <https://artificialanalysis.ai/>.
- [41] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo and Y. Iwasawa, ‘Large language models are zero-shot reasoners,’ in *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho and A. Oh, Eds., 2022.
- [42] T. B. Brown et al., ‘Language models are few-shot learners,’ in *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan and H. Lin, Eds., 2020.
- [43] J. Wei et al., ‘Chain-of-thought prompting elicits reasoning in large language models,’ in *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho and A. Oh, Eds., 2022.
- [44] Y. Zhang, Y. Yuan and A. C.-C. Yao, *Meta prompting for ai systems*, 2025. arXiv: [2311.11482](https://arxiv.org/abs/2311.11482) [cs.AI].
- [45] D. Hulbert, *Using tree-of-thought prompting to boost chatgpt’s reasoning*, <https://github.com/dave1010/tree-of-thought-prompting>, May 2023. DOI: [10.5281/ZENODO.10323452](https://doi.org/10.5281/ZENODO.10323452).
- [46] J. Long, ‘Large language model guided tree-of-thought,’ *CoRR*, vol. abs/2305.08291, 2023. DOI: [10.48550/ARXIV.2305.08291](https://doi.org/10.48550/ARXIV.2305.08291). arXiv: [2305.08291](https://arxiv.org/abs/2305.08291).
- [47] S. Yao et al., ‘Tree of thoughts: Deliberate problem solving with large language models,’ in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt and S. Levine, Eds., 2023.
- [48] T. Sharma, M. Fragkoulis and D. Spinellis, ‘Does your configuration code smell?’ In *Proceedings of the 13th International Conference on Mining Software Repositories, MSR 2016, Austin, TX, USA, May 14-22, 2016*, M. Kim, R. Robbes and C. Bird, Eds., ACM, 2016, pp. 189–200. DOI: [10.1145/2901739.2901761](https://doi.org/10.1145/2901739.2901761).